



Universidad de las Américas Puebla (UDLAP)

Licenciatura en Ciencia de Datos

## **Actividad 9: Semáforo binario y contador**

Andrea Hernandez Huerta

ID: 175523

Heriberto Espino Montelongo

ID: 175199

Profesor: Dr. Victor Giovanni Morales Murillo

Curso: Programación Concurrente

Fecha de entrega: 2 de octubre de 2026

# Índice general

|                                               |            |
|-----------------------------------------------|------------|
| <b>Índice de figuras</b>                      | <b>III</b> |
| <b>Índice de cuadros</b>                      | <b>IV</b>  |
| <b>1. Semáforo binario: cuenta bancaria</b>   | <b>1</b>   |
| 1.1. Entradas . . . . .                       | 1          |
| 1.2. Procesamiento . . . . .                  | 1          |
| 1.3. Estructura de clases . . . . .           | 2          |
| 1.4. Utilidad compartida . . . . .            | 2          |
| 1.5. Conceptos utilizados . . . . .           | 3          |
| 1.6. Prueba de escritorio . . . . .           | 4          |
| 1.7. Salida esperada y comprobación . . . . . | 5          |
| 1.8. Evidencia de la implementación . . . . . | 6          |
| <b>2. Semáforo contador: impresoras</b>       | <b>9</b>   |
| 2.1. Entradas . . . . .                       | 9          |
| 2.2. Procesamiento . . . . .                  | 9          |
| 2.3. Estructura de clases . . . . .           | 10         |
| 2.4. Prueba de escritorio . . . . .           | 10         |
| 2.5. Salida esperada y comprobación . . . . . | 11         |
| 2.6. Evidencia de la implementación . . . . . | 13         |
| <b>3. Comparación y conclusión</b>            | <b>16</b>  |
| 3.1. Comparación . . . . .                    | 16         |
| 3.2. Conclusión . . . . .                     | 16         |
| <b>Referencias</b>                            | <b>18</b>  |

# Índice de figuras

|      |                                                                                                                         |    |
|------|-------------------------------------------------------------------------------------------------------------------------|----|
| 1.1. | Clase Java de utilidades compartidas por ambos ejercicios. . . . .                                                      | 3  |
| 1.2. | Ejecución del semáforo binario: saldo final de \$1300 y máximo de un hilo modificando el saldo simultáneamente. . . . . | 5  |
| 1.3. | Implementación de BinarySemaphore, P() y V(). . . . .                                                                   | 6  |
| 1.4. | Implementación de OperacionBancaria y sus datos compartidos. . . . .                                                    | 7  |
| 1.5. | Sección crítica de la operación bancaria y actualización del saldo. . . . .                                             | 7  |
| 1.6. | Creación de los cinco hilos y comprobación del semáforo binario. . . . .                                                | 8  |
| 2.1. | Ejecución del semáforo contador: máximo de tres hilos imprimiendo simultáneamente, igual al límite permitido. . . . .   | 12 |
| 2.2. | Implementación de CountingSemaphore, P() y V(). . . . .                                                                 | 13 |
| 2.3. | Implementación de TareaImpresion y control del máximo simultáneo. . . . .                                               | 14 |
| 2.4. | Creación de las ocho tareas y ejecución del semáforo contador. . . . .                                                  | 15 |

# Índice de cuadros

|      |                                                                                       |    |
|------|---------------------------------------------------------------------------------------|----|
| 1.1. | Responsabilidad de las clases de la parte 1. . . . .                                  | 2  |
| 1.2. | Prueba de escritorio del semáforo binario con dos hilos. . . .                        | 4  |
| 2.1. | Responsabilidad de las clases de la parte 2. . . . .                                  | 10 |
| 2.2. | Prueba de escritorio del semáforo contador con dos impresoras y cuatro hilos. . . . . | 11 |
| 3.1. | Comparación entre semáforo binario y semáforo contador. .                             | 16 |

# 1. Semáforo binario: cuenta bancaria

**Instrucción.** Crear una cuenta bancaria compartida con saldo inicial de \$1000 y cinco hilos que realicen depósitos y retiros. Utilizar un semáforo binario inicializado en 1. Cada hilo debe solicitar el permiso, consultar y modificar el saldo y liberar el permiso. Se debe comprobar que solamente un hilo modifique el saldo a la vez y realizar una prueba de escritorio con dos hilos.

## 1.1 Entradas

La cuenta bancaria inicia con un saldo de \$1000. Los cinco hilos comparten la misma instancia de `CuentaBancaria` y la misma instancia de `BinarySemaphore`. El semáforo se crea con `true`, equivalente a un único permiso disponible.

Las operaciones utilizadas en la ejecución son tres depósitos de \$200, \$300 y \$50, y dos retiros de \$150 y \$100. Por ello, el saldo final esperado es

$$1000 + 200 - 150 + 300 - 100 + 50 = 1300.$$

## 1.2 Procesamiento

La clase `BinarySemaphore` mantiene una variable booleana `value`. Cuando su valor es `true`, el permiso está disponible; cuando es `false`, el permiso está ocupado.

La operación `P()` es un método `synchronized`. Mientras el permiso esté ocupado, el hilo ejecuta `Util.mywait(this)`. La espera libera temporalmente el monitor para permitir que otro hilo ejecute `V()`. Al obtener el permiso, `P()` cambia el valor a `false`.

La operación `V()` también es `synchronized`: cambia el valor a `true` y ejecuta `notify()` para despertar a uno de los hilos bloqueados.

La clase `CuentaBancaria` no utiliza `synchronized`; el acceso al saldo queda protegido externamente por el semáforo. Cada `OperacionBancaria` ejecuta `P()` antes de entrar a la sección crítica y `V()` en la sección de salida.

## 1.3 Estructura de clases

**Cuadro 1.1:** Responsabilidad de las clases de la parte 1.

| Clase             | Responsabilidad                                                                                                                       | Uso de <code>synchronized</code>                                    |
|-------------------|---------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------|
| Principal         | Crea la cuenta, el semáforo y los cinco hilos; utiliza <code>start()</code> y <code>join()</code> .                                   | No lo requiere.                                                     |
| BinarySemaphore   | Controla el único permiso mediante <code>P()</code> y <code>V()</code> .                                                              | <code>P()</code> y <code>V()</code> son <code>synchronized</code> . |
| CuentaBancaria    | Almacena el saldo y realiza depósitos y retiros.                                                                                      | No es necesario si el acceso está protegido por el semáforo.        |
| OperacionBancaria | Define la sección crítica ejecutada por cada hilo y utiliza <code>P()</code> antes y <code>V()</code> después de modificar la cuenta. | No lo requiere.                                                     |

## 1.4 Utilidad compartida

La clase `Util` concentra las esperas bloqueadas y las pausas usadas por ambos ejercicios. Su método `mywait()` ejecuta `wait()` sobre el monitor reci-

bido y conserva la interrupción del hilo; `mysleep()` permite simular el tiempo de una operación sin convertir la espera en un ciclo activo.



```

09 Semáforo binario y contador > code > java > Util.java > ...
1  public final class Util {
2      private Util() {
3      }
4
5      public static void mywait(Object monitor) {
6          boolean interrumpido = false;
7
8          while (true) {
9              try {
10                 monitor.wait();
11                 break;
12             } catch (InterruptedException excepcion) {
13                 interrumpido = true;
14             }
15         }
16
17         if (interrumpido) {
18             Thread.currentThread().interrupt();
19         }
20     }
21
22     public static void mysleep(long milisegundos) {
23         try {
24             Thread.sleep(milisegundos);
25         } catch (InterruptedException excepcion) {
26             Thread.currentThread().interrupt();
27         }
28     }
29 }

```

Figura 1.1: Clase Java de utilidades compartidas por ambos ejercicios.

## 1.5 Conceptos utilizados

El semáforo binario realiza exclusión mutua porque existe un solo permiso. Si un hilo ya lo obtuvo, los demás se bloquean en `P()` hasta que el hilo que se encuentra en la sección crítica ejecute `V()`. A diferencia de la espera activa estudiada con Test-and-Set, la operación de espera utiliza `wait()`, por lo que el

hilo bloqueado libera el monitor y no permanece consultando continuamente el valor del semáforo.

En las diapositivas,  $P()$  corresponde a solicitar un permiso, también llamada  $acquire()$ ,  $wait()$  o  $down()$ .  $V()$  corresponde a liberar un permiso, también llamada  $release()$ ,  $signal()$  o  $up()$ .

## 1.6 Prueba de escritorio

Para la prueba se consideran dos hilos. El hilo A deposita \$200 y el hilo B retira \$150. El semáforo inicia disponible y el saldo inicia en \$1000.

**Cuadro 1.2:** Prueba de escritorio del semáforo binario con dos hilos.

| Paso | Acción               | Semáforo | Saldo | Resultado                                    |
|------|----------------------|----------|-------|----------------------------------------------|
| 0    | Estado inicial       | 1        | 1000  | Permiso disponible.                          |
| 1    | Hilo A ejecuta $P()$ | 0        | 1000  | A entra a la sección crítica.                |
| 2    | Hilo B ejecuta $P()$ | 0        | 1000  | B se bloquea porque el permiso está ocupado. |
| 3    | A deposita \$200     | 0        | 1200  | Solo A modifica la cuenta.                   |
| 4    | A ejecuta $V()$      | 1        | 1200  | Se libera el permiso y B es despertado.      |
| 5    | B completa $P()$     | 0        | 1200  | B obtiene el permiso.                        |
| 6    | B retira \$150       | 0        | 1050  | Solo B modifica la cuenta.                   |
| 7    | B ejecuta $V()$      | 1        | 1050  | El permiso queda disponible.                 |

## 1.7 Salida esperada y comprobación

El programa muestra cuándo cada hilo entra a la sección crítica, el saldo antes y después de la operación y el saldo final. Las cinco operaciones producen un saldo esperado de \$1300. La comprobación adicional debe reportar un máximo de un hilo modificando el saldo al mismo tiempo.

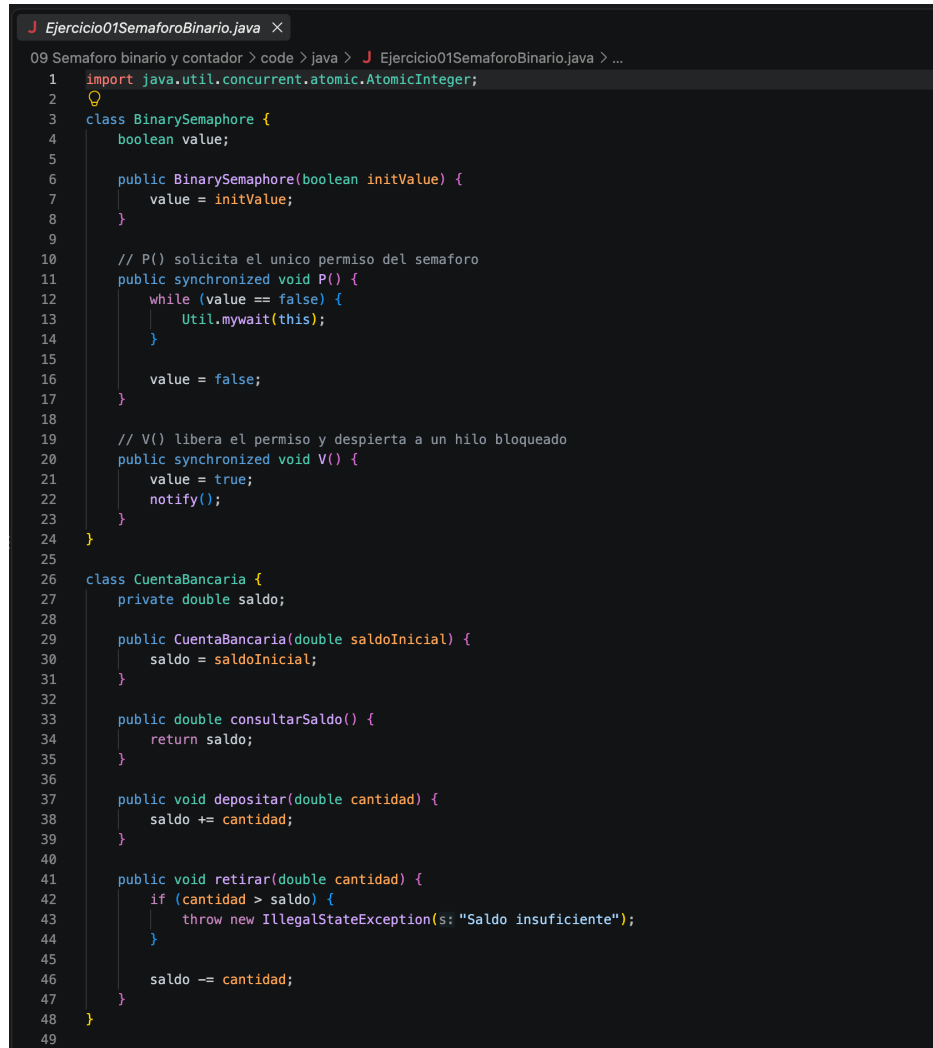
```

heri .../09 Semaforo binario y contador/code/java main !? v26.0.2 base 17:56
java Ejercicio01SemaforoBinario.java
=== Semaforo binario: cuenta bancaria ===
Saldo inicial: $1000.00
Hilo A entra | saldo antes = $1000.00
Hilo A deposita $200.00
Hilo A sale | saldo despues = $1200.00
Hilo B entra | saldo antes = $1200.00
Hilo B retira $150.00
Hilo B sale | saldo despues = $1050.00
Hilo C entra | saldo antes = $1050.00
Hilo C deposita $300.00
Hilo C sale | saldo despues = $1350.00
Hilo D entra | saldo antes = $1350.00
Hilo D retira $100.00
Hilo D sale | saldo despues = $1250.00
Hilo E entra | saldo antes = $1250.00
Hilo E deposita $50.00
Hilo E sale | saldo despues = $1300.00
Saldo final esperado: $1300.00
Saldo final obtenido: $1300.00
Máximo de hilos modificando el saldo al mismo tiempo: 1
heri .../09 Semaforo binario y contador/code/java main !? v26.0.2 base 17:56
java Ejercicio01SemaforoBinario.java
=== Semaforo binario: cuenta bancaria ===
Saldo inicial: $1000.00
Hilo A entra | saldo antes = $1000.00
Hilo A deposita $200.00
Hilo A sale | saldo despues = $1200.00
Hilo B entra | saldo antes = $1200.00
Hilo B retira $150.00
Hilo B sale | saldo despues = $1050.00
Hilo C entra | saldo antes = $1050.00
Hilo C deposita $300.00
Hilo C sale | saldo despues = $1350.00
Hilo D entra | saldo antes = $1350.00
Hilo D retira $100.00
Hilo D sale | saldo despues = $1250.00
Hilo E entra | saldo antes = $1250.00
Hilo E deposita $50.00
Hilo E sale | saldo despues = $1300.00
Saldo final esperado: $1300.00
Saldo final obtenido: $1300.00
Máximo de hilos modificando el saldo al mismo tiempo: 1

```

**Figura 1.2:** Ejecución del semáforo binario: saldo final de \$1300 y máximo de un hilo modificando el saldo simultáneamente.

## 1.8 Evidencia de la implementación



```
J Ejercicio01SemaforoBinario.java X
09 Semaforo binario y contador > code > java > J Ejercicio01SemaforoBinario.java > ...
1  import java.util.concurrent.atomic.AtomicInteger;
2
3  class BinarySemaphore {
4      boolean value;
5
6      public BinarySemaphore(boolean initialValue) {
7          value = initialValue;
8      }
9
10     // P() solicita el unico permiso del semaforo
11     public synchronized void P() {
12         while (value == false) {
13             Util.mywait(this);
14         }
15
16         value = false;
17     }
18
19     // V() libera el permiso y despierta a un hilo bloqueado
20     public synchronized void V() {
21         value = true;
22         notify();
23     }
24 }
25
26 class CuentaBancaria {
27     private double saldo;
28
29     public CuentaBancaria(double saldoInicial) {
30         saldo = saldoInicial;
31     }
32
33     public double consultarSaldo() {
34         return saldo;
35     }
36
37     public void depositar(double cantidad) {
38         saldo += cantidad;
39     }
40
41     public void retirar(double cantidad) {
42         if (cantidad > saldo) {
43             throw new IllegalStateException(s: "Saldo insuficiente");
44         }
45
46         saldo -= cantidad;
47     }
48 }
49
```

Figura 1.3: Implementación de BinarySemaphore, P() y V().

```
J Ejercicio01SemaforoBinario.java X
09 Semaforo binario y contador > code > java > J Ejercicio01SemaforoBinario.java > ...
50 class OperacionBancaria implements Runnable {
51     public enum Tipo {
52         DEPOSITO,
53         RETIRO
54     }
55
56     // estos contadores solo comprueban el resultado y no controlan el acceso
57     private static final AtomicInteger HILOS_EN_SECCION_CRITICA = new AtomicInteger(initialValue: 0);
58     private static final AtomicInteger MAXIMO_SIMULTANEO = new AtomicInteger(initialValue: 0);
59
60     private final BinarySemaphore mutex;
61     private final CuentaBancaria cuenta;
62     private final Tipo tipo;
63     private final double cantidad;
64
65     public OperacionBancaria(BinarySemaphore mutex, CuentaBancaria cuenta,
66         Tipo tipo, double cantidad) {
67         this.mutex = mutex;
68         this.cuenta = cuenta;
69         this.tipo = tipo;
70         this.cantidad = cantidad;
71     }
72
73     @Override
74     public void run() {
```

Figura 1.4: Implementación de OperacionBancaria y sus datos compartidos.

```
J Ejercicio01SemaforoBinario.java X
09 Semaforo binario y contador > code > java > J Ejercicio01SemaforoBinario.java > ...
50 class OperacionBancaria implements Runnable {
73     @Override
74     public void run() {
75         // P() equivale a solicitar el permiso con acquire()
76         mutex.P();
77
78         try {
79             int simultaneos = HILOS_EN_SECCION_CRITICA.incrementAndGet();
80             MAXIMO_SIMULTANEO.accumulateAndGet(simultaneos, Math::max);
81
82             if (simultaneos > 1) {
83                 throw new IllegalStateException(
84                     s: "Mas de un hilo entro a la seccion critica");
85             }
86
87             double saldoAntes = cuenta.consultarSaldo();
88             String nombre = Thread.currentThread().getName();
89
90             System.out.printf(format: "%s entra | saldo antes = %.2f%n",
91                 nombre, saldoAntes);
92
93             // hace visible que los demas hilos deben esperar el permiso
94             Util.mysleep(milisegundos: 120);
95
96             if (tipo == Tipo.DEPOSITO) {
97                 cuenta.depositar(cantidad);
98                 System.out.printf(format: "%s deposita %.2f%n", nombre, cantidad);
99             } else {
100                 cuenta.retirar(cantidad);
101                 System.out.printf(format: "%s retira %.2f%n", nombre, cantidad);
102             }
103
104             System.out.printf(format: "%s sale | saldo despues = %.2f%n",
105                 nombre, cuenta.consultarSaldo());
106         } finally {
107             HILOS_EN_SECCION_CRITICA.decrementAndGet();
108             // V() equivale a liberar el permiso con release()
109             mutex.V();
110         }
111     }
112
113     public static int getMaximoSimultaneo() {
114         return MAXIMO_SIMULTANEO.get();
115     }
116 }
117 }
```

Figura 1.5: Sección crítica de la operación bancaria y actualización del saldo.

```
J Ejercicio01SemaforoBinario.java X
09 Semaforo binario y contador > code > java > J Ejercicio01SemaforoBinario.java > ...
118 public class Ejercicio01SemaforoBinario {
119     public static void main(String[] args) throws InterruptedException {
120         CuentaBancaria cuenta = new CuentaBancaria(saldoInicial: 1000.0);
121         BinarySemaphore mutex = new BinarySemaphore(initValue: true);
122
123         Thread[] hilos = {
124             new Thread(new OperacionBancaria(
125                 mutex, cuenta, OperacionBancaria.Tipo.DEPOSITO, cantidad: 200.0), name: "Hilo A"),
126             new Thread(new OperacionBancaria(
127                 mutex, cuenta, OperacionBancaria.Tipo.RETIRO, cantidad: 150.0), name: "Hilo B"),
128             new Thread(new OperacionBancaria(
129                 mutex, cuenta, OperacionBancaria.Tipo.DEPOSITO, cantidad: 300.0), name: "Hilo C"),
130             new Thread(new OperacionBancaria(
131                 mutex, cuenta, OperacionBancaria.Tipo.RETIRO, cantidad: 100.0), name: "Hilo D"),
132             new Thread(new OperacionBancaria(
133                 mutex, cuenta, OperacionBancaria.Tipo.DEPOSITO, cantidad: 50.0), name: "Hilo E")
134         };
135
136         System.out.println(x: "=== Semaforo binario: cuenta bancaria ===");
137         System.out.printf(format: "Saldo inicial: %.2f%n", cuenta.consultarSaldo());
138
139         for (Thread hilo : hilos) {
140             hilo.start();
141         }
142
143         for (Thread hilo : hilos) {
144             hilo.join();
145         }
146
147         System.out.printf(format: "Saldo final esperado: $1300.00%n");
148         System.out.printf(format: "Saldo final obtenido: %.2f%n", cuenta.consultarSaldo());
149         System.out.println("Maximo de hilos modificando el saldo al mismo tiempo: "
150             + OperacionBancaria.getMaximoSimultaneo());
151     }
152 }
153
```

Figura 1.6: Creación de los cinco hilos y comprobación del semáforo binario.

## 2. Semáforo contador: impresoras

**Instrucción.** Simular un laboratorio con tres impresoras y ocho hilos que desean imprimir. Utilizar un semáforo contador inicializado en 3. Cada hilo debe solicitar una impresora, indicar que comienza a imprimir, esperar un tiempo para simular la impresión, indicar que termina y liberar la impresora. Se debe verificar que nunca existan más de tres hilos imprimiendo simultáneamente y realizar una prueba de escritorio con dos impresoras y cuatro hilos.

### 2.1 Entradas

El recurso está formado por tres impresoras equivalentes. El programa crea un semáforo contador con valor inicial 3 y ocho hilos que comparten el mismo semáforo. Cada tarea mantiene ocupada una impresora durante un intervalo breve para hacer visible la concurrencia en la salida.

### 2.2 Procesamiento

La clase `CountingSemaphore` mantiene un entero `value`, como en el pseudocódigo mostrado en clase. En `P()` se decrementa el valor. Si el resultado es negativo, el hilo ejecuta una espera bloqueada. Un valor negativo representa que existen hilos esperando un recurso.

En `V()` se incrementa el valor. Si después del incremento el valor es menor o igual que cero, existe al menos un hilo esperando y se ejecuta `notify()` para despertar a uno. Tanto `P()` como `V()` son métodos `synchronized`.

Cada hilo solicita una impresora con `P()`, realiza la simulación de impresión y finalmente devuelve el permiso con `V()`. La pausa usada para simular

la impresión no constituye el mecanismo de sincronización; únicamente representa el tiempo durante el cual el recurso permanece ocupado.

## 2.3 Estructura de clases

**Cuadro 2.1:** Responsabilidad de las clases de la parte 2.

| Clase             | Responsabilidad                                                                                           | Uso de <b>synchronized</b>                                          |
|-------------------|-----------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------|
| Principal         | Crea el semáforo con tres permisos y los ocho hilos; utiliza <code>start()</code> y <code>join()</code> . | No lo requiere.                                                     |
| CountingSemaphore | Mantiene el número de permisos y controla la espera mediante <code>P()</code> y <code>V()</code> .        | <code>P()</code> y <code>V()</code> son <code>synchronized</code> . |
| TareaImpresion    | Solicita una impresora, simula su utilización y la libera al terminar.                                    | No lo requiere.                                                     |

## 2.4 Prueba de escritorio

La prueba utiliza dos impresoras y cuatro hilos A, B, C y D. Los dos primeros hilos obtienen un recurso. Los siguientes disminuyen el contador por debajo de cero y quedan bloqueados. Cuando A y B liberan sus impresoras, C y D son despertados de uno en uno.

**Cuadro 2.2:** Prueba de escritorio del semáforo contador con dos impresoras y cuatro hilos.

| Paso | Acción             | Semáforo | Cola  | Resultado                                    |
|------|--------------------|----------|-------|----------------------------------------------|
| 0    | Estado inicial     | 2        | Vacía | Dos impresoras disponibles.                  |
| 1    | Hilo A ejecuta P() | 1        | Vacía | A utiliza una impresora.                     |
| 2    | Hilo B ejecuta P() | 0        | Vacía | B utiliza la otra impresora.                 |
| 3    | Hilo C ejecuta P() | -1       | C     | C se bloquea.                                |
| 4    | Hilo D ejecuta P() | -2       | C, D  | D se bloquea.                                |
| 5    | Hilo A ejecuta V() | -1       | D     | C despierta y utiliza la impresora liberada. |
| 6    | Hilo B ejecuta V() | 0        | Vacía | D despierta y utiliza la impresora liberada. |
| 7    | Hilo C ejecuta V() | 1        | Vacía | Queda una impresora disponible.              |
| 8    | Hilo D ejecuta V() | 2        | Vacía | Quedan dos impresoras disponibles.           |

## 2.5 Salida esperada y comprobación

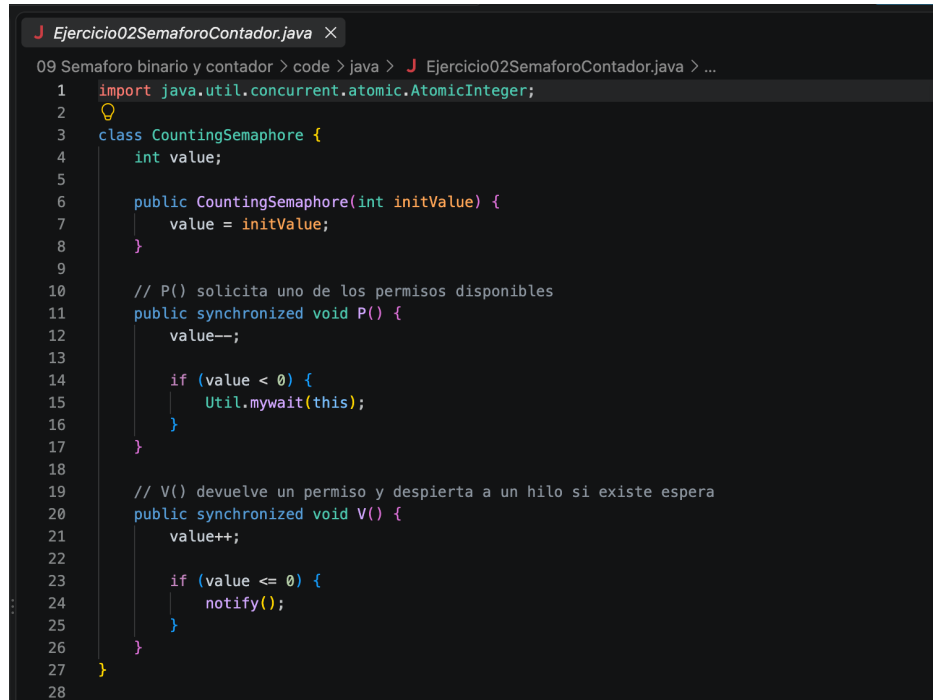
La salida identifica cuándo inicia y termina cada hilo y muestra cuántos hilos se encuentran imprimiendo. El orden exacto puede cambiar entre ejecu-

ciones porque depende de la planificación de los hilos. La propiedad que sí debe mantenerse es que el máximo observado nunca sea mayor que 3.

```
heri ~/09 Semaforo binario y contador/code/java ▶ main !? v26.0.2 base 17:58
java Ejercicio02SemaforoContador.java
=== Semaforo contador: tres impresoras ===
Hilo 2 comienza a imprimir | simultaneos = 2
Hilo 3 comienza a imprimir | simultaneos = 3
Hilo 1 comienza a imprimir | simultaneos = 1
Hilo 1 termina de imprimir
Hilo 2 termina de imprimir
Hilo 4 comienza a imprimir | simultaneos = 2
Hilo 5 comienza a imprimir | simultaneos = 3
Hilo 3 termina de imprimir
Hilo 6 comienza a imprimir | simultaneos = 3
Hilo 4 termina de imprimir
Hilo 5 termina de imprimir
Hilo 7 comienza a imprimir | simultaneos = 3
Hilo 8 comienza a imprimir | simultaneos = 3
Hilo 6 termina de imprimir
Hilo 7 termina de imprimir
Hilo 8 termina de imprimir
Máximo de hilos imprimiendo al mismo tiempo: 3
Límite permitido por el semaforo: 3
heri ~/09 Semaforo binario y contador/code/java ▶ main !? v26.0.2 base 17:58
java Ejercicio02SemaforoContador.java
=== Semaforo contador: tres impresoras ===
Hilo 3 comienza a imprimir | simultaneos = 3
Hilo 2 comienza a imprimir | simultaneos = 2
Hilo 1 comienza a imprimir | simultaneos = 1
Hilo 2 termina de imprimir
Hilo 1 termina de imprimir
Hilo 4 comienza a imprimir | simultaneos = 2
Hilo 3 termina de imprimir
Hilo 5 comienza a imprimir | simultaneos = 3
Hilo 6 comienza a imprimir | simultaneos = 3
Hilo 5 termina de imprimir
Hilo 6 termina de imprimir
Hilo 7 comienza a imprimir | simultaneos = 3
Hilo 4 termina de imprimir
Hilo 8 comienza a imprimir | simultaneos = 3
Hilo 7 termina de imprimir
Hilo 8 termina de imprimir
Máximo de hilos imprimiendo al mismo tiempo: 3
Límite permitido por el semaforo: 3
```

**Figura 2.1:** Ejecución del semáforo contador: máximo de tres hilos imprimiendo simultáneamente, igual al límite permitido.

## 2.6 Evidencia de la implementación



```
J Ejercicio02SemaforoContador.java X
09 Semaforo binario y contador > code > java > J Ejercicio02SemaforoContador.java > ...
1  import java.util.concurrent.atomic.AtomicInteger;
2
3  class CountingSemaphore {
4      int value;
5
6      public CountingSemaphore(int initValue) {
7          value = initValue;
8      }
9
10     // P() solicita uno de los permisos disponibles
11     public synchronized void P() {
12         value--;
13
14         if (value < 0) {
15             Util.mywait(this);
16         }
17     }
18
19     // V() devuelve un permiso y despierta a un hilo si existe espera
20     public synchronized void V() {
21         value++;
22
23         if (value <= 0) {
24             notify();
25         }
26     }
27 }
28
```

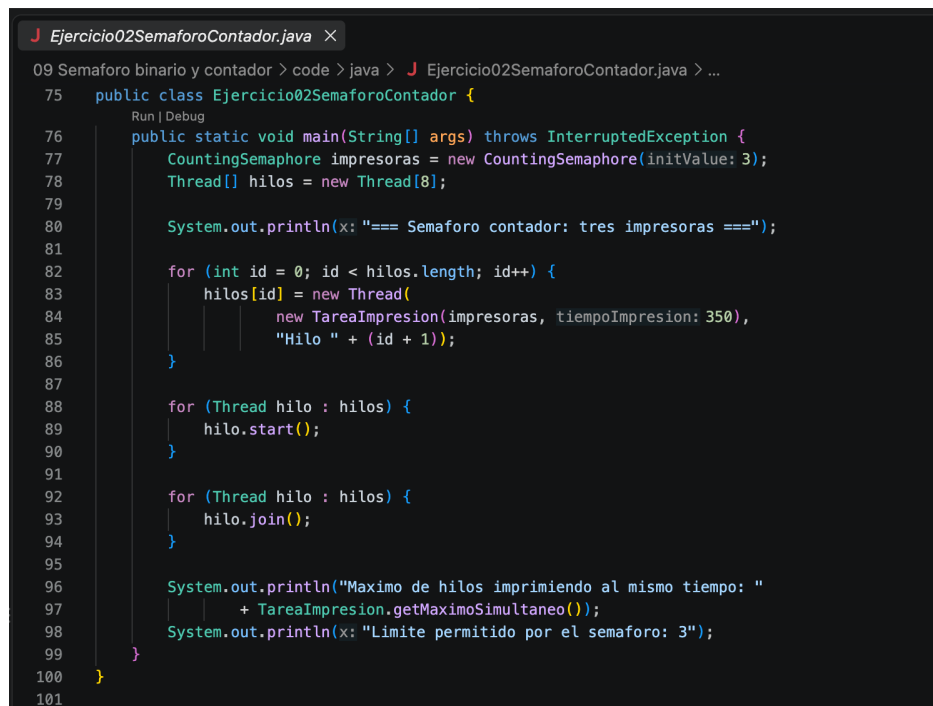
Figura 2.2: Implementación de CountingSemaphore, P() y V().

```

J Ejercicio02SemaforoContador.java X
09 Semaforo binario y contador > code > java > J Ejercicio02SemaforoContador.java > ...
29 class TareaImpresion implements Runnable {
30     // estos contadores solo comprueban el resultado y no conceden permisos
31     private static final AtomicInteger IMPRIMIENDO = new AtomicInteger(initialValue: 0);
32     private static final AtomicInteger MAXIMO_SIMULTANEO = new AtomicInteger(initialValue: 0);
33
34     private final CountingSemaphore impresoras;
35     private final long tiempoImpresion;
36
37     public TareaImpresion(CountingSemaphore impresoras, long tiempoImpresion) {
38         this.impresoras = impresoras;
39         this.tiempoImpresion = tiempoImpresion;
40     }
41
42     @Override
43     public void run() {
44         // P() equivale a solicitar una impresora con acquire()
45         impresoras.P();
46
47         try {
48             int simultaneos = IMPRIMIENDO.incrementAndGet();
49             MAXIMO_SIMULTANEO.accumulateAndGet(simultaneos, Math::max);
50
51             if (simultaneos > 3) {
52                 throw new IllegalStateException(
53                     s: "Hay mas de tres hilos imprimiendo al mismo tiempo");
54             }
55
56             String nombre = Thread.currentThread().getName();
57             System.out.println(nombre + " comienza a imprimir | simultaneos = "
58                 + simultaneos);
59
60             Util.mysleep(tiempoImpresion);
61
62             System.out.println(nombre + " termina de imprimir");
63         } finally {
64             IMPRIMIENDO.decrementAndGet();
65             // V() equivale a liberar la impresora con release()
66             impresoras.V();
67         }
68     }
69
70     public static int getMaximoSimultaneo() {
71         return MAXIMO_SIMULTANEO.get();
72     }
73 }
74

```

Figura 2.3: Implementación de TareaImpresion y control del máximo simultáneo.



```
J Ejercicio02SemaforoContador.java X
09 Semaforo binario y contador > code > java > J Ejercicio02SemaforoContador.java > ...
75 public class Ejercicio02SemaforoContador {
    Run | Debug
76     public static void main(String[] args) throws InterruptedException {
77         CountingSemaphore impresoras = new CountingSemaphore(initValue: 3);
78         Thread[] hilos = new Thread[8];
79
80         System.out.println(x: "=== Semaforo contador: tres impresoras ===");
81
82         for (int id = 0; id < hilos.length; id++) {
83             hilos[id] = new Thread(
84                 new TareaImpresion(impresoras, tiempoImpresion: 350),
85                 "Hilo " + (id + 1));
86         }
87
88         for (Thread hilo : hilos) {
89             hilo.start();
90         }
91
92         for (Thread hilo : hilos) {
93             hilo.join();
94         }
95
96         System.out.println("Maximo de hilos imprimiendo al mismo tiempo: "
97             + TareaImpresion.getMaximoSimultaneo());
98         System.out.println(x: "Limite permitido por el semaforo: 3");
99     }
100 }
101 }
```

**Figura 2.4:** Creación de las ocho tareas y ejecución del semáforo contador.

## 3. Comparación y conclusión

### 3.1 Comparación

**Cuadro 3.1:** Comparación entre semáforo binario y semáforo contador.

| Aspecto                            | Semáforo binario                     | Semáforo contador                     |
|------------------------------------|--------------------------------------|---------------------------------------|
| Valor inicial en la actividad      | 1 (true)                             | 3                                     |
| Permisos disponibles inicialmente  | 1                                    | 3                                     |
| Recurso controlado                 | Cuenta bancaria                      | Tres impresoras                       |
| Número máximo de hilos simultáneos | 1                                    | 3                                     |
| Cuándo se bloquea un hilo          | Cuando el único permiso está ocupado | Cuando no queda un permiso disponible |

### 3.2 Conclusión

Los dos semáforos coordinan hilos mediante permisos, pero representan problemas diferentes. El semáforo binario mantiene un solo permiso y por ello se utiliza para exclusión mutua sobre un recurso compartido, como el saldo de la cuenta. El semáforo contador mantiene varios permisos y permite que una cantidad limitada de hilos utilice simultáneamente recursos equivalentes, como las tres impresoras.

En ambos casos, `P()` solicita un permiso y puede bloquear al hilo, mientras que `V()` devuelve un permiso y puede despertar a un hilo que espera. La diferencia principal está en la cantidad de permisos que representa el semáforo. Las pruebas de escritorio muestran esta diferencia: el semáforo binario obliga a que los accesos a la cuenta ocurran uno por uno, mientras que el semáforo

contador permite hasta tres impresiones concurrentes sin superar el número de recursos disponibles.

# Bibliografía

- [1] V. G. Morales Murillo, “Semáforo, semáforo binario y semáforo contador,” 2026, diapositivas del curso Programación Concurrente, Universidad de las Américas Puebla.